

Aluno: _____ Nota: _____

Avaliação 2 – 2025.2

1. (Valor 2) Considere o código abaixo. Você deve complementar o código para que após o retorno da função "alocaEspaco", o ponteiro p deva estar apontando para um espaço de memória alocado dentro da função com o tamanho relativo ao valor de num. Não pode usar variável global, nem modificar o tipo da função. Caso precise, faça um rascunho fora, mas a sua resposta tem que ser colocada dentro da figura abaixo.

```
1  #include <stdio.h>
2
3  void alocaEspaco(                ){
4
5
6
7
8  }
9
10 int main(){
11     int *p = NULL;
12     int num;
13     scanf("%d", &num);
14
15
16     alocaEspaco(                );
17
18
19
20 }
21
```

2. (Valor 5) Considere o trabalho 2. Um aluno resolveu pedir a IA para modelar o trabalho 2. O resultado foi uma lista encadeada que guarda tanto a estrutura principal quanto as auxiliares. Para isso, foi definido duas informações no nó da lista encadeada: **info** e **tam**. O dado **info** é usado de duas formas: 1) quando o nó da lista encadeada representa um elemento da estrutura principal (trabalho 2), o valor representa a quantidade de elementos que têm na estrutura auxiliar (trabalho 2); 2) nos outros casos, **info** representa o número existente na estrutura auxiliar. O dado **tam** é a segunda informação do nó. Ele só é útil nos casos que o elemento da lista encadeada representa o nó da estrutura principal. **tam** guarda o tamanho da estrutura auxiliar. A figura a seguir mostra um exemplo da estrutura.

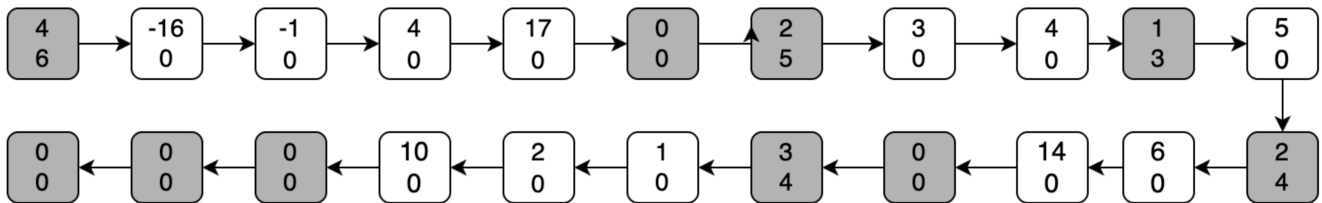


Figura 1 - Lista Encadeada

Na figura, foi destacado de cinza os nós que representam as informações da estrutura principal. Porém, todos os nós são do mesmo tipo. Observe que o primeiro nó tem dois valores: **info** = 4, que quer dizer a quantidade de elementos existentes na estrutura auxiliar conectada a essa posição; e **tam** = 6, que representa o tamanho dessa estrutura auxiliar. A Figura 2 abaixo é a estrutura do trabalho 2 resultante, para o exemplo da Figura 1 acima. Como no trabalho 2, a estrutura principal guarda a quantidade de elementos e o tamanho da estrutura auxiliar.

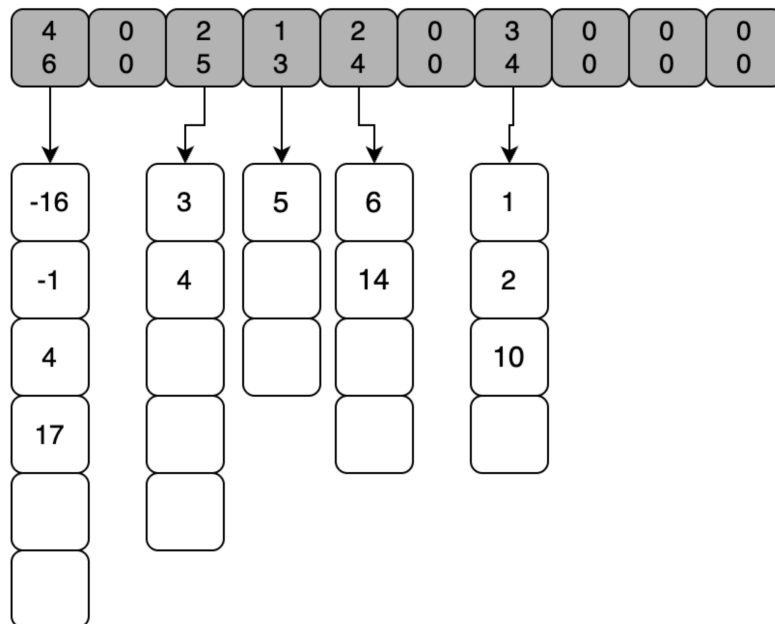


Figura 2 - Estrutura do trabalho 2

Faça:

- a. (Valor 0.5) Defina a(s) estrutura(s) necessária(s) para representar a lista encadeada (Figura 1);
- b. (Valor 0.5) Defina a(s) estrutura(s) necessária(s) para representar a lista principal (Figura 2);
- c. (Valor 1.0) Na função *main*, **defina** a variáveis necessárias para acessar a lista encadeada (Figura 1) e a estrutura principal local (não deve ser global como no trabalho 2) (Figura 2), e **chame** uma função de nome *pegaValores*; obs. Ao voltar da chamada da função a estrutura principal local deve estar preenchida com os valores corretos;
- d. (Valor 3.0) a função *pegaValores* deve ler os dados da lista encadeada (Figura 1) e criar toda a estrutura resultante (Figura 2).

Observações

- defina tipo de retorno e parâmetros adequados, quando necessário
- não pode utilizar nenhuma função pronta do trabalho